

GATE Complexity — Complete Solutions

All 15 Questions with Step-by-Step Working

SECTION 1 — GATE-Style Recurrences & Complexity

Q1 (GATE 2021 Style)

$T(n) = T(n/2) + T(2n/5) + 7n$, $T(0) = 1$ (fractions: $1/2 + 2/5 = 9/10$)

- Fraction of problem covered per level: $1/2 + 2/5 = 9/10 < 1$
- Work at each level forms a geometric series: $7n + 7n(9/10) + 7n(9/10)^2 + \dots$
- Sum = $7n * 1/(1 - 9/10) = 7n * 10 = 70n$
- Since the series converges, total work is linear.

✓ Answer: $T(n) = \Theta(n)$

Q2 (GATE 2021 Style)

$T(n) = aT(n/b) + f(n) \Rightarrow T(n) = \Theta(n \log n)$

- By Master Theorem Case 2: $T(n) = \Theta(n^{\log_b a} \log n)$
- For this to equal $\Theta(n \log n)$, we need $n^{\log_b a} = n \Rightarrow \log_b(a) = 1 \Rightarrow a = b$
- Then $f(n)$ must be $\Theta(n)$ to satisfy Case 2 condition.

✓ Answer: Condition: $a = b$ and $f(n) = \Theta(n)$

Q3 (GATE 2024 Style)

Single pass over array of size N , comparing adjacent elements only.

- One pass = exactly $N-1$ comparisons (no nesting, no recursion).
- Work is directly proportional to N .

✓ Answer: $T(n) = \Theta(n)$

Q4 (GATE 2015 Style)

Evaluate: $1^3 + 2^3 + 3^3 + \dots + n^3$

- Closed form: $[n(n+1)/2]^2 = n^4/4 + \text{lower-order terms}$
- Leading term is n^4 , so tight bound is $\Theta(n^4)$
- $\Theta(n^4)$ ✓ — exact tight bound
- $\Theta(n^5)$ ✗ — too loose for Theta
- $O(n^5)$ ✓ — n^4 is certainly $O(n^5)$
- $\Omega(n^3)$ ✓ — n^4 is certainly $\Omega(n^3)$

✓ Answer: $\Theta(n^4)$, $O(n^5)$, $\Omega(n^3)$ are all correct

Q5 (GATE 2015 Style)

$fun1(n)$: outer i -loop (1 to n), inner j -loop (n down by halves $\rightarrow p$), inner k -loop (doubles until $k \geq p$)

- j -loop: $j = n, n/2, \dots, 1 \Rightarrow p = \lfloor \log_2 n \rfloor$ iterations
- k -loop: $k = 1, 2, 4, \dots$ until $k \geq p \Rightarrow \lfloor \log_2 \log_2 n \rfloor$ iterations
- Outer i -loop runs $n-1$ times.
- Total $q = (n-1) * \log(\log n)$

✓ Answer: $q = \Theta(n \log \log n)$

Q6 (Classic GATE)

```
for(i=n; i>0; i/=2) { for(j=0; j<i; j++) { } }
```

- Outer loop: $i = n, n/2, n/4, \dots, 1 \Rightarrow \log n$ iterations
- Inner loop runs i times each: total = $n + n/2 + n/4 + \dots + 1$
- Geometric series with ratio $1/2$: sum = $n * (1/(1-1/2)) = 2n$

✓ Answer: $\Theta(n)$

Q7 (Classic GATE)

```
for(i=1; i<=n; i++) { for(j=1; j<=i; j++) { } }
```

- Inner j runs from 1 to i for each i .
- Total = $1 + 2 + 3 + \dots + n = n(n+1)/2$

✓ Answer: $\Theta(n^2)$

SECTION 2 — Recursion Trees & Fibonacci

Q8 (Recursion)

`void fun(int n) { if(n<=1) return; fun(n/2); fun(n/2); }`

- Recurrence: $T(n) = 2T(n/2) + O(1)$
- Master Theorem: $a=2, b=2, f(n)=1$
- $n^{\log_b a} = n^{\log_2 2} = n^1 = n$
- $f(n) = O(n^{\log_b a - \epsilon}) \Rightarrow$ Case 1 applies

✓ Answer: $\Theta(n)$

Q9 (Fibonacci Pattern)

`int fib(int n) { if(n<=1) return n; return fib(n-1)+fib(n-2); }`

- Recurrence: $T(n) = T(n-1) + T(n-2)$
- Each call branches into 2 slightly smaller calls; depth is n .
- Number of nodes grows like Fibonacci sequence itself.
- Fibonacci grows as ϕ^n where $\phi = (1+\sqrt{5})/2 \approx 1.618$ (golden ratio)

✓ Answer: $\Theta(\phi^n) \approx \Theta(2^n)$ (exponential)

Q10 (GATE Favourite)

Arrange in increasing order: $n^{3/2}, n \log n, n^{\log_2 n}, 2^n$

- Test at $n = 64 = 2^6$:
- $n \log n = 64 * 6 = 384$
- $n^{3/2} = 64^{1.5} = 512$
- $n^{\log_2 n} = 64^6 = 2^{36} \approx 6.8 \times 10^{10}$
- $2^n = 2^{64} \approx 1.8 \times 10^{19}$

✓ Answer: $n \log n < n^{3/2} < n^{\log n} < 2^n$

SECTION 3 — PSU-Level Recurrences (Q11–Q15)

Q11 (PSU Level)

$$T(n) = T(n-1) + 1$$

- Unroll: $T(n) = T(n-1)+1 = T(n-2)+2 = \dots = T(0)+n$
- $T(0) = 1$ (base case), so $T(n) = n + 1$

✓ Answer: $\Theta(n)$

Q12 (PSU Level)

$$T(n) = T(n-1) + n$$

- Unroll: $T(n) = n + (n-1) + (n-2) + \dots + 1$
- This is the triangular sum $= n(n+1)/2$

✓ Answer: $\Theta(n^2)$

Q13 (PSU Level)

$$T(n) = 2T(n-1) + 1$$

- Unroll: $T(n) = 2T(n-1)+1 = 4T(n-2)+3 = 2^k T(n-k) + (2^k-1)$
- At $k = n$: $T(n) = 2^n T(0) + (2^n-1) = 2^n + 2^n - 1$
- Dominant term is 2^n

✓ Answer: $\Theta(2^n)$

Q14 (PSU Level)

$$T(n) = T(n/2) + 1$$

- Classic binary-search recurrence.
- n is halved each step until reaching 1 — takes $\log_2 n$ steps.
- Adding 1 at each step: $T(n) = \log_2 n + 1$

✓ Answer: $\Theta(\log n)$

Q15 (PSU Level)

for($i=1$; $i \leq n$; $i*=2$) { *for*($j=1$; $j \leq n$; $j++$) { } }

- Outer loop: $i = 1, 2, 4, \dots, n \Rightarrow \log n$ iterations
- Inner loop: always runs n times (independent of i)
- Total $= \log n * n$

✓ Answer: $\Theta(n \log n)$

SUMMARY TABLE

Q#	Type	Answer
Q1	Recurrence (split)	$\Theta(n)$
Q2	Master Theorem condition	$a = b$, $f(n) = \Theta(n)$
Q3	Single pass	$\Theta(n)$
Q4	Sum of cubes	$\Theta(n^4)$, $O(n^4)$, $\Omega(n^3)$

Q5	Nested loops (log log)	$\Theta(n \log \log n)$
Q6	Geometric series loop	$\Theta(n)$
Q7	Triangular loop	$\Theta(n^2)$
Q8	Two recursive halves	$\Theta(n)$
Q9	Fibonacci recursion	$\Theta(\phi^n)$
Q10	Growth order ranking	$n \log n < n^{3/2} < n^{\log n} < 2^n$
Q11	$T(n)=T(n-1)+1$	$\Theta(n)$
Q12	$T(n)=T(n-1)+n$	$\Theta(n^2)$
Q13	$T(n)=2T(n-1)+1$	$\Theta(2^n)$
Q14	$T(n)=T(n/2)+1$	$\Theta(\log n)$
Q15	Double loop (log * n)	$\Theta(n \log n)$

All solutions use standard asymptotic notation: Θ (tight), O (upper), Ω (lower). Master Theorem cases and geometric series are the primary tools used throughout.